

# Pundarikaksh N. Tripathi

📞 +91 8957987974 | ✉️ pundarikaksh.dev@gmail.com | 🔗 linkedin.com/in/pundarikakshnarayantripathi | 🌐 github.com/PundarikakshNTripathi

## Experience

**FlyRank AI** | Causal ML, X-Learner, Random Forest, scikit-learn, DuckDB, Hugging Face Datasets, Python

Remote

Machine Learning Engineering Intern

July 2026 – September 2026

- Built a refresh-opportunity ranker on 30,000 anonymized pages: dropped label-derived columns (`trend_pct`, `trend_direction`), trained a Random Forest on a client-grouped 70/30 split, raising Precision@50 from 0.56 (hand-written rule) to 0.70.
- Quantified leakage: random-split P@50 0.94 vs 0.70 under client-grouped holdout (0.24 optimism gap); reframed claims as observational.
- Capstone (mentor-accepted): X-Learner estimating the CATE of a content refresh (random-forest learners, decile Qini on a client-grouped holdout); ran DuckDB `hf://` Parquet queries over the full 79.8M-row warehouse.
- Built an action playbook (reason codes, no-go list); documented the unobserved-confounding limit and the need for an A/B test.

## Projects

🔗 **TernixEngine: Ternary GEMM Kernels, CPU (AVX2) and GPU (CUDA)** | C++20, AVX2, CUDA `cp.async`, PyBind11, GoogleTest, CMake, 1.58-bit

- Wrote branch-free 1.58-bit ternary GEMM in C++20/AVX2: 2-bit packed weights decoded in-register (`_mm256_sr1v_epi32`, `mask`, `shift`) and applied to int8 activations with `_mm256_sign_epi32`; a 4-row register-blocked variant reuses each decoded weight vector.
- Benchmarked 512<sup>3</sup> on one pinned P-core (i7-14650HX, seeded data): 14.4 ms AVX2 and 8.5 ms register-blocked (31 G ops/s) vs 86.8 ms scalar (10.2×) and 445 ms naive (52×).
- Wrote a 64<sup>3</sup>-tiled CUDA kernel with a two-stage `cp.async` 16-byte double buffer, bank-conflict-free padded shared rows and 4×4 outputs per thread: 21.3 ms at 4096<sup>3</sup> (6.5 TOPS) on an RTX 5060, 5.6× a naive GPU kernel, bit-exact vs CPU.
- Verified all kernels against an independent integer GEMM (GoogleTest, pytest/NumPy via PyBind11, GPU-vs-CPU exactness); CI builds Linux/Windows and compiles CUDA for sm\_80–90. Kernel library in active research and development.

🔗 [Source: github.com/Quiet-Intelligence/TernixEngine](https://github.com/Quiet-Intelligence/TernixEngine)

🔗 **VoltaSplat: Differentiable CUDA Gaussian-Splatting Rasterizer** | CUDA C++, PyTorch ATen, CUB, C++17, autograd, pybind11

- Built a tile-based differentiable 3D Gaussian Splatting rasterizer in CUDA/C++ behind a PyTorch `autograd.Function`: EWA projection, per-tile (`tile<<32|depth`) keys, CUB radix sort, shared-memory alpha compositing with early termination.
- Derived the backward pass by hand: reverse-order compositing gradients (`atomicAdd`) and an analytic chain through the conic, Jacobian and viewport transform to 3D means and covariances.
- Matches an independent PyTorch reference renderer (images 2e-7, gradients 1e-6 relative); the reference passes float64 `gradcheck`.
- On an RTX 5060: 1M Gaussians at 800×800 render in 6.1 ms (164 FPS), forward+backward in 27 ms; fitting 4,000 Gaussians lifts PSNR 11.0→31.2 dB (SSIM 0.98) in 800 steps.

🔗 [Source: github.com/PundarikakshNTripathi/VoltaSplat](https://github.com/PundarikakshNTripathi/VoltaSplat)

🔗 **nanoDist: Data Parallelism and ZeRO-2 from Scratch** | Python, NumPy, TCP sockets, ring all-reduce, ZeRO-2, FP16, Hydra, Optuna, W&B

- Implemented ring all-reduce (reduce-scatter + all-gather) in-process and across OS processes over TCP sockets, with bucketed gradients; per-rank traffic verified at 2(N−1)/N of the buffer.
- Built ZeRO stage 2 on a from-scratch ring all-reduce: gradient reduce-scatter, Adam on each rank's 1/N parameter shard with 1/N optimizer state, parameter all-gather; matches single-process Adam (2 and 4 processes).
- Added FP16 compute on FP32 master weights with loss scaling, gradient accumulation and activation checkpointing; analytic accounting for a 1024→4096→1024 MLP: 105.97→33.84 MB per worker (−68%).
- 27 tests (Adam equivalence, finite-difference gradients, multi-process TCP); they exposed a crash in the checkpointing path, which I fixed; CI runs ruff, mypy and pytest.

🔗 [Source: github.com/PundarikakshNTripathi/nanoDist](https://github.com/PundarikakshNTripathi/nanoDist)

🔗 **Aegis: Behavioral Governance for Autonomous Agents (OpenAI Buildweek)** | Go, C, eBPF LSM, seccomp, AWS Cedar, SQLite, LinUCB, RL

- Designed and built (one-week sprint; teammates helped with presentation and live testing) a Linux control plane for AI coding agents: C eBPF LSM hooks on `file_open`, `socket_connect` and `bprm_check_security` feed a ring buffer read by a Go daemon.
- Compiled stored denials (a Cedar `forbid` subset) into BPF map entries for in-kernel blocking; added a seccomp user-notification exec gate, a Bubble Tea TUI and an offline LinUCB bandit (FN −5, FP −1).
- Validated live on Linux and WSL2 in a recorded demo: blocked the post-exploitation payload of BunnyHijack (a disputed Bun package-manager PoC) at the syscall layer, plus other unauthorised commands.

🔗 [Source: github.com/Quiet-Intelligence/aegis](https://github.com/Quiet-Intelligence/aegis)

## Achievements

- **3rd on the public and private leaderboards (89,000+ teams), Amazon ML Challenge 2026** at the last check; macro-F0.5 0.991762. Led planning, coordination and review of a 3-person team's entity-resolution pipeline (1.73M businesses vs ≈10M noisy records): 13 lexical retrievers + e5 kNN, LightGBM A→B→C cascade with cross-encoder blend, calibration, expected-F0.5 set selection.
  - Diagnosed the validation/leaderboard gap (0.9896 vs 0.981) as unseen-country shift (France appears only in test); fixed with per-country IDF and pseudo-labels, adopting changes only if leave-one-country-out improved both ways.
- **2nd place, Masai × NoBroker HackArena 2025**: CivicAgent, an AI civic-complaint triage platform (Gemini Vision classification, APScheduler SLA tracking with email escalation, Supabase/PostGIS); proof of concept, no longer hosted.

## Technical Skills

**Languages:** Python, C++20, CUDA C++, PTX (inline asm), Go, C, SQL, TypeScript, Bash

**ML Frameworks:** PyTorch, JAX, Transformers (Hugging Face), ONNX, Triton, scikit-learn, LightGBM, EconML, DoWhy

**Systems and Infra:** eBPF (LSM), AVX2 SIMD, CUDA (`cp.async`, CUB), DDP / ZeRO / ring all-reduce, gRPC, Docker, Kubernetes, CMake, GitHub Actions

## Education

**Babu Banarasi Das University**

Lucknow, UP

B.Tech, Computer Science and Engineering (Artificial Intelligence)

Expected August 2027

- **Coursework:** Computer Architecture, Digital Logic Design, Data Structures, Algorithm Design and Analysis, Linear Algebra, Operating Systems.
- **Self-study:** modern C++, CUDA, ML systems, causal inference.